

**ТЕХНОЛОГИЧНО УЧИЛИЩЕ ЕЛЕКТРОННИ СИСТЕМИ
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ**

КУРСОВ ПРОЕКТ

Тема: Състезателна бесеница

Ученик:
Борис Стрински

Ръководител:
Кристиян Йочев

1. Въведение

Настоящият документ описва реализацията на 'Състезателна бесеница' — мрежова игра за двама играча, базирана на класическата игра бесеница. Двата играча избират думи, които другият трябва да отгатне. Победителят е този, допуснал по-малко грешни букви. При равен брой грешки играта завършва с равенство.

1.1. Структура на проекта

Решението се състои от следните файлове:

Файл	Предназначение
<code>server.c</code>	TCP сървър — управлява свързването, играта и финалния резултат
<code>client.c</code>	TCP клиент — осигурява потребителски интерфейс за играча
<code>game.c</code>	Библиотека с игровата логика (предоставена, непроменяна)
<code>game.h</code>	Заглавен файл на библиотеката (предоставен, непроменян)
<code>Makefile</code>	Инструкции за компилиране на <code>hangman-server</code> и <code>hangman-client</code>

1.2. Компилиране и стартиране

Проектът се компилира с командата:

```
$ make
```

Стартиране на сървъра:

```
$ ./hangman-server <port>
```

Стартиране на клиент:

```
$ ./hangman-client <host> <port> <word>
```

Пример за пълна игра

Терминал 1: `$ ./hangman-server 8080`

Терминал 2: `$ ./hangman-client 127.0.0.1 8080 hello`

Терминал 3: `$ ./hangman-client 127.0.0.1 8080 world`

2. Комуникационен протокол

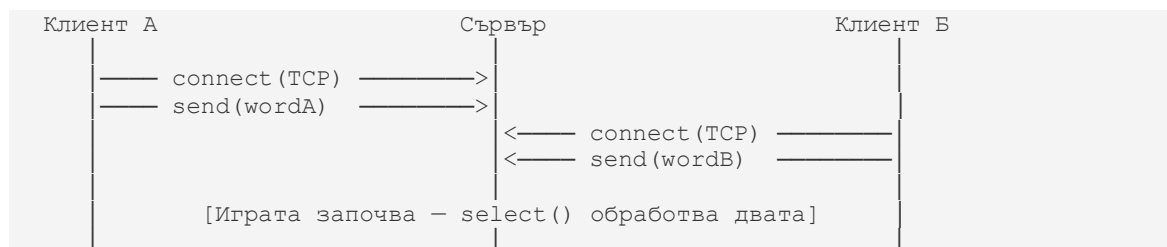
Протоколът е текстово базиран и работи върху TCP. Всички съобщения са на прости ASCII редове, завършващи с \n. Комуникацията се разделя на три фази.

2.1. Обща таблица на съобщенията

Посока и формат	Описание и кога се изпраща
C → S: <word>	Клиентът изпраща избраната дума веднага след свързване (raw bytes, без \n)
S → C: Word: h_ll_\n	Текущото маскирано състояние на думата (отгатнатите букви са видими)
S → C: Incorrect guesses: a, i\n	Списък с неправилно отгатнатите букви след всяка стъпка
C → S: <letter>\n	Едно предположение от играча (малка буква + нов ред)
S → C: YOU WIN! :) \n (или др.)	Финален резултат — изпраща се веднъж при край на играта
S → C: Your incorrect guesses: ... \n	Броят грешки на текущия играч (финален блок)
S → C: Opponent's incorrect guesses: \n	Броят грешки на противника (финален блок)

2.2. Фаза 1 — Свързване и размяна на думи

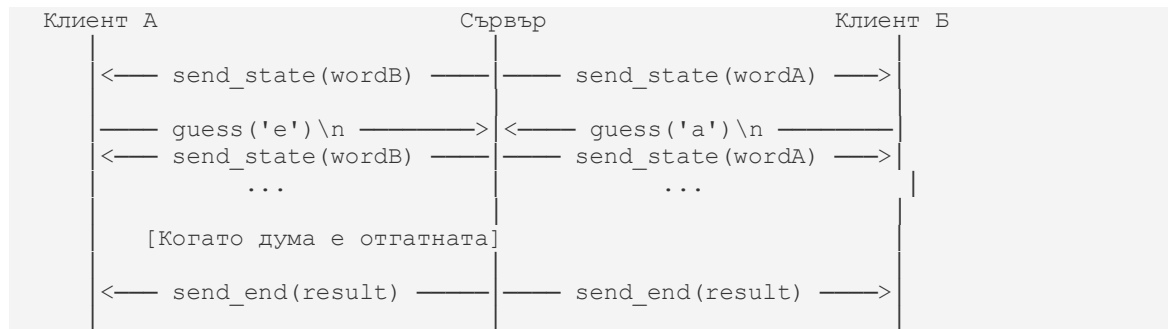
Двата клиента се свързват към сървъра независимо един от друг. Сървърът изчаква и двамата да се свържат, преди да стартира играта. Веднага след установяване на TCP връзката, всеки клиент изпраща избраната от него дума — тя ще бъде думата, която противникът трябва да отгатне.



Думата се изпраща като суров поток от байтове. Сървърът използва специална функция `read_word()`, която чете символи докато не срещне разделител или прекъсване. Всяка буква се нормализира до малка с `normalize()`.

2.3. Фаза 2 — Игрови цикъл

След получаване на двете думи, сървърът инициализира `game[0]` с думата на Клиент Б (за Клиент А) и `game[1]` с думата на Клиент А (за Клиент Б). Изпраща се началното състояние и на двамата. Оттук играта тече едновременно за двата клиента чрез `select()`.



Клиентът чете два реда (`Word: ...` и `Incorrect guesses: ...`), проверява дали думата е напълно отгатната (няма символ `'_'`), и ако не е — чете буква от `stdin` и я изпраща обратно на сървъра.

2.4. Фаза 3 — Край на играта

Когато и двамата играчи са отгатнали думите си (`done[0] && done[1]`), сървърът сравнява броя на неправилните предположения и изпраща финалния блок от три реда на всеки клиент:

```
YOU WIN! :)                                (или: You Lose! :( / Tie :/)  
Your incorrect guesses: a, i, c  
Opponent's incorrect guesses: e, f
```

Клиентът разпознава края на играта, когато в реда `Word:` няма символ `_` — тогава вместо да чака ново предположение, изпълнява `read_end()`, която чете финалния тризреден блок и го отпечатва.

2.5. Антимамене

Сигурност на протокола

Сървърът НИКОГА не изпраща оригиналната дума на клиента.

Клиентът получава единствено маскирания вариант (`Word: h_ll_`).

`game[i]` съдържа думата на играч (1-i), т.е. всеки играч вижда само своя прогрес.

Дори при прехващане на трафика, противникът не може да узнае думата.

3. Работа на сървъра (server.c)

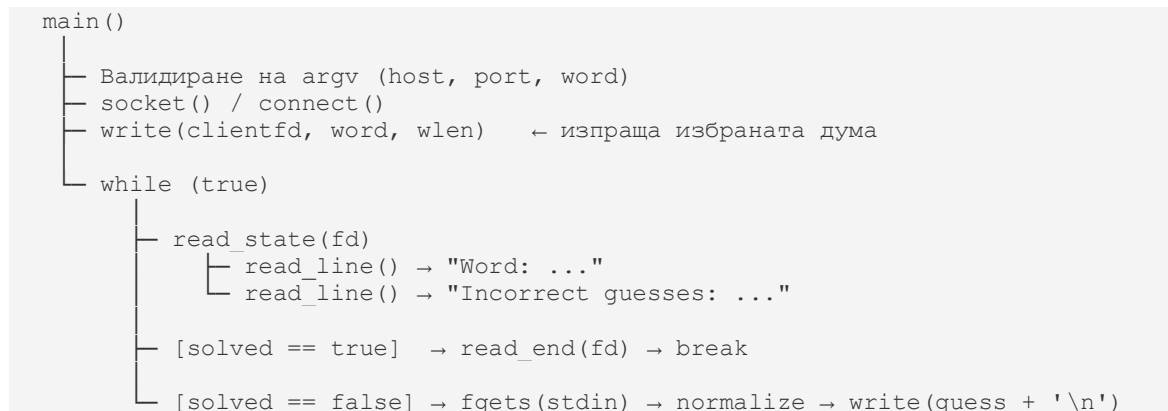
```
// game[i] = думата, избрана от играч (1-i)
secret_word_init_from_c_string(&game[0], word_buf[1]); // А отгатва думата на Б
secret_word_init_from_c_string(&game[1], word_buf[0]); // Б отгатва думата на А
```

3.2.4. Четене на думата при свързване

Функцията `read_word()` чете символи един по един. Първият символ се чете блокиращо (`read()`), а следващите — с `MSG_DONTWAIT`, за да не блокира ако клиентът не изпрати повече. Това позволява коректно четене когато думата пристига като един TCP сегмент.

4. Работа на клиента (client.c)

4.1. Поток на изпълнение



4.2. Валидиране на входните параметри

При стартиране клиентът извършва следните проверки на аргументите:

- Портът трябва да съдържа само цифри (`isdigit()`)
- IP адресът трябва да съдържа само цифри и точки
- Думата трябва да съдържа само букви след нормализация с `normalize()`

4.3. Игрови цикъл

Централната логика на клиента е в безкраен цикъл, управляван от функцията `read_state()`. Тя чете двата реда (Word: и Incorrect guesses:) и връща:

Върната стойност	Значение и действие на клиента
1	Думата не е отгатната — клиентът чете буква от stdin и я изпраща
0	Думата е отгатната — клиентът извиква <code>read_end()</code> и излиза от цикъла
-1	Връзката е затворена — клиентът прекъсва цикъла

4.4. Разпознаване на отгатната дума

Клиентът разпознава, че думата е отгатната чрез проверка дали редът `Word:` съдържа символа `_`. Ако не съдържа долни черти, всички букви са разкрити.

```
bool solved = (strstr(line, "_") == NULL) && (strncmp(line, "Word:", 5) == 0);
```


4.5. Нормализация на буквите

Преди изпращане, въведената буква се нормализира с `normalize()` (конвертира главни букви в малки). Само валидни букви (`is_letter()`) се изпращат — невалидни символи се игнорират с `continue`.

5. Игрова библиотека (game.c / game.h)

Библиотеката е предоставена и не е модифицирана. Тя реализира цялата игрова логика чрез две основни структури от данни.

5.1. letter_set_t

Множество от букви, реализирано като 32-битов битфийлд (`uint32_t bitfield`). Бит 0 = 'a', бит 1 = 'b', ..., бит 25 = 'z'. Тази структура позволява $O(1)$ операции за добавяне, премахване и проверка.

Функция	Описание
<code>letter_set_add(set, c)</code>	Добавя буква към множеството
<code>letter_set_contains(set, c)</code>	Проверява дали буква е в множеството
<code>letter_set_size(set)</code>	Връща броя на буквите в множеството
<code>letter_set_intersect/union</code>	Сечение / обединение на две множества

5.2. secret_word_t

Представява тайната дума с нейното текущо отгатнато състояние. Съдържа следните полета:

- `word` — указател към символния масив на думата
- `revealed_bitfield` — битмаска: 1 = буква е отгатната, 0 = скрита
- `word_length` — дължина на думата
- `incorrect_guesses` — `letter_set_t` с всички грешни букви
- `all_guesses` — `letter_set_t` с всички опитани букви

Функция	Описание
<code>secret_word_init_from_c_string()</code>	Инициализира структурата от C-стринг
<code>secret_word_guess(word, c)</code>	Обработка предположение; връща CORRECT/INCORRECT/ALREADY_GUESSED
<code>secret_word_letter_at(word, i, &c)</code>	Връща буквата на позиция i (ако е разкрита)
<code>secret_word_is_solved(word)</code>	Връща true ако всички букви са отгатнати
<code>secret_word_incorrect_guess_count()</code>	Брой неправилните предположения
<code>secret_word_free(word)</code>	Освобождава динамичната памет

5.3. Определяне на победителя

След края на играта, сървърът извиква `secret_word_incorrect_guess_count()` за двата обекта и сравнява резултатите. Играчът с по-малко грешки печели. При равенство се изпраща `Tie :/`.

```
size_t wrong[2];
for (int i = 0; i < 2; i++)
    wrong[i] = secret_word_incorrect_guess_count(&game[i]);

if      (wrong[0] < wrong[1]) { outcome[0]="YOU WIN! :)"; outcome[1]="You
Lose! :("; }
else if (wrong[1] < wrong[0]) { outcome[0]="You Lose! :("; outcome[1]="YOU
WIN! :)"; }
else                                     { outcome[0]="Tie :/";
outcome[1]="Tie :/"; }
```

Резюме на архитектурата

- Единичен процес без нишки — паралелност чрез `select()` мултиплексиране на I/O
- Текстов TCP протокол — прост, лесен за дебъгване с `netcat` или `telnet`
- Антимамене — сървърът никога не изпраща оригиналната дума на клиента
- `SO_REUSEADDR` — позволява бързо рестартиране след тест
- Нормализация на входа — клиентът и сървърът обработват малки и главни букви
- Чиста управление на памет — `secret_word_free()` при край на играта